AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Kamil Kwiczal

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Przykład 4

2. Instalacja projektu 4

3. Zastosowanie listy dwukierunkowej 4

4. Sposób działania algorytmu 4

5. Przykład działania listy dwukierunkowej 5

6. Opis wykorzystanych narzędzi 5

7. Wykorzystywane narzędzia i technologie 5

8. Struktura projektu 6

9. Zastosowane wzorce projektowe 6

9.1. Wzorzec Factory 6

9.2. Wzorzec Iterator 6

10. Zarządzanie pamięcią 7

11. Kontrola wersji za pomocą Git i GitHub 7

12. Generowanie dokumentacji za pomocą Doxygen 7

13. Implementacja klasy Node 7

14. Inicjalizacja i zwalnianie pamięci 8

15. Dodawanie elementów do listy 8

16. Wyświetlanie elementów listy 9

17. Usuwanie elementów 9

18. Obsługa iteratora 9

19. Czyszczenie listy 10

Literatura 12

Spis rysunków 12

Spis tabel **13**

Spis listingów **14**

Ogólne określenie wymagań Program powinien implementować listę dwukierunkową w języku C++, działającą na stercie, z wykorzystaniem wzorców projektowych Factory i Iterator. Powinien umożliwiać dodawanie, usuwanie, wyświetlanie i czyszczenie elementów listy oraz poruszanie się po niej w obu kierunkach. Każda klasa powinna znajdować się w osobnym pliku, a jej działanie należy przetestować w funkcji main. Projekt powinien zawierać dokumentację wygenerowaną za pomocą Doxygen, dokumentację projektową oraz historię pracy z wykorzystaniem GitHuba.

1. Przykład

Lista dwukierunkowa może być wykorzystana do przechowywania i przeglądania liczb, z możliwością przechodzenia do następnego i poprzedniego elementu. Użytkownik może dodawać i usuwać liczby z wybranego miejsca oraz wyświetlać je w kolejności od początku do końca lub odwrotnie.

2. Instalacja projektu

Aby pobrać projekt z serwisu GitHub, należy uruchomić terminal i wpisać następujące polecenie:

```
1 git clone https://github.com/kjkwiczal-byte/ans-lista-dwukierunkowa
    .git
```

Listing 1. Klonowanie repozytorium

Analiza problemu

3. Zastosowanie listy dwukierunkowej

Lista dwukierunkowa jest strukturą danych wykorzystywaną w programowaniu do przechowywania i organizowania elementów, które można przeglądać w obu kierunkach. Jej zaletą jest możliwość dynamicznego dodawania i usuwania elementów bez konieczności przesuwania pozostałych danych.

4. Sposób działania algorytmu

Lista dwukierunkowa składa się z węzłów, z których każdy przechowuje wartość oraz dwa wskaźniki: na poprzedni i następny element. Pierwszy węzeł nie posiada

poprzednika, a ostatni nie posiada następnika. Dodawanie elementu polega na utworzeniu nowego węzła i odpowiednim ustawieniu wskaźników, natomiast usuwanie wymaga połączenia ze sobą sąsiednich węzłów. Przeglądanie listy odbywa się przez przechodzenie między węzłami, a czyszczenie polega na usunięciu wszystkich elementów i zwolnieniu zajmowanej pamięci.

5. Przykład działania listy dwukierunkowej

Przykładowo, lista może przechowować liczby 15, 20 i 30. Początkowo lista jest pusta: `NULL`

Po dodaniu liczb na koniec listy otrzymujemy: `NULL <- 15 <-> 20 <-> 30 -> NULL`

Przechodząc od początku, program wyświetli liczby w kolejności: 15, 20, 30. Przechodząc od końca, wyświetli: 30, 20, 15.

6. Opis wykorzystanych narzędzi

Git to system kontroli wersji, który umożliwia zapisywanie historii zmian w projekcie, porównywanie wersji kodu oraz przywracanie wcześniejszych stanów programu. GitHub to internetowa platforma służąca do przechowywania repozytoriów Git i współpracy nad projektami. Doxygen to narzędzie służące do automatycznego generowania dokumentacji na podstawie komentarzy umieszczonych w kodzie źródłowym. LaTeX to system przygotowywania dokumentów, szczególnie przydatny przy tworzeniu dokumentacji technicznej i naukowej. Projektowanie

7. Wykorzystywane narzędzia i technologie

Projekt został opracowany w języku C++, który umożliwia tworzenie struktur danych oraz zarządzanie pamięcią dynamiczną. Program realizuje działanie listy dwukierunkowej przechowującej elements w pamięci sterty.

Podczas realizacji projektu wykorzystywane są następujące narzędzia:

- **C++** – język programowania wykorzystany do implementacji listy dwukierunkowej oraz wzorców projektowych.
- **Git** – system kontroli wersji umożliwiający śledzenie zmian w kodzie.

- **GitHub** – platforma umożliwiająca przechowywanie repozytorium zdalnego.
- **Doxygen** – narzędzie automatycznie generujące dokumentację.
- **LaTeX** – system składu tekstu wykorzystywany do przygotowania dokumentacji projektowej.

8. Struktura projektu

Projekt został podzielony na osobne pliki, aby oddzielić odpowiedzialności poszczególnych klas. Ułatwia to rozwijanie programu, testowanie jego działania oraz utrzymanie kodu.

W projekcie wykorzystywane są następujące klasy i pliki:

- **Node** – reprezentuje pojedynczy węzeł listy.
- **NodeFactory** – odpowiada za tworzenie nowych węzłów.
- **ListIterator** – umożliwia przechodzenie po elementach listy.
- **DoublyLinkedList** – implementuje listę dwukierunkową.
- **main** – zawiera funkcję główną programu.

9. Zastosowane wzorce projektowe

9.1. Wzorzec Factory

Wzorzec Factory został zastosowany do tworzenia węzłów listy. Zamiast umieszczać całą logikę tworzenia obiektów bezpośrednio w klasie listy, program korzysta z klasy `NodeFactory`. Takie rozwiązanie oddziela tworzenie węzłów od operacji wykonywanych na liście.

9.2. Wzorzec Iterator

Wzorzec Iterator służy do poruszania się po elementach listy. Umożliwia odczytywanie kolejnych elementów oraz przechodzenie do elementu poprzedniego bez konieczności umieszczania całej logiki iteracji w klasie głównej programu.

10. Zarządzanie pamięcią

Elementy listy są tworzone dynamicznie w pamięci sterty. Podczas dodawania elementu należy odpowiednio zaktualizować powiązania między węzłami. Przy usuwaniu elementu konieczne jest zachowanie poprawności połączeń pozostałych węzłów oraz zwolnienie pamięci zajmowanej przez usuwany obiekt.

11. Kontrola wersji za pomocą Git i GitHub

Podczas realizacji projektu wykorzystywany jest system Git, który pozwala zapisywać kolejne etapy pracy nad programem. W ramach projektu Git służy również do sprawdzania działania operacji związanych z historią zmian, w tym rozwiązywania konfliktów (merge conflict) oraz resetowania historii.

12. Generowanie dokumentacji za pomocą Doxygen

Doxygen jest wykorzystywany do automatycznego generowania dokumentacji na podstawie kodu źródłowego oraz komentarzy. Plik konfiguracyjny określa między innymi formaty dokumentacji i katalog wyjściowy. Implementacja Implementacja projektu została wykonana w języku C++. Program wykorzystuje listę dwukierunkową, której elementy są przechowywane w pamięci dynamicznej. Kod projektu został podzielony na pliki nagłówkowe `.h` oraz pliki źródłowe `.cpp`.

13. Implementacja klasy Node

Klasa Node reprezentuje pojedynczy element listy. Przechowuje wartość liczbową w polu data oraz dwa wskaźniki: na poprzedni i następny element.

```
#ifndef NODE_H
#define NODE_H
class Node {
public:
    int data;
    Node* prev;
    Node* next;
    Node(int val);
};
```

```
#endif
```

Listing 2. Deklaracja klasy Node

```
#include "Node.h"
Node::Node(int val) : data(val), prev(nullptr), next(nullptr) {}
```

Listing 3. Konstruktor klasy Node

14. Inicjalizacja i zwalnianie pamięci

Konstruktor klasy `DoublyLinkedList` ustawia wskaźniki na `nullptr`. Destruktor wywołuje metodę `clear()`, która usuwa wszystkie elementy listy i zwalnia zajmowaną przez nie pamięć.

```
DoublyLinkedList::DoublyLinkedList() : head(nullptr), tail(nullptr)
    , size(0) {}

DoublyLinkedList::~DoublyLinkedList() {
    clear();
}
```

Listing 4. Konstruktor i destruktor klasy listy

15. Dodawanie elementów do listy

Metoda `push_front()` dodaje nowy element na początku listy. W tym celu tworzy węzeł za pomocą metody `NodeFactory::createNode()`.

```
void DoublyLinkedList::push_front(int data) {
    Node* newNode = NodeFactory::createNode(data);
    if (head == nullptr) {
        head = tail = newNode;
    } else {
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }
    size++;
}
```

Listing 5. Dodawanie elementu na początku listy

16. Wyświetlanie elementów listy

Do wyświetlania zawartości listy od początku do końca służy metoda `print()`. Przechodzi ona przez kolejne węzły, aż do momentu napotkania wartości nullptr.

```
void DoublyLinkedList::print() {
    Node* current = head;
    std::cout << "Lista (od poczatku): ";
    while (current != nullptr) {
        std::cout << current->data << " ";
        current = current->next;
    }
    std::cout << std::endl;
}
```

Listing 6. Wyświetlanie listy w przód

17. Usuwanie elementów

Metoda `pop_front()` usuwa pierwszy element listy. Na początku sprawdza, czy lista nie jest pusta.

```
void DoublyLinkedList::pop_front() {
    if (head == nullptr) return;
    Node* temp = head;
    if (head == tail) {
        head = tail = nullptr;
    } else {
        head = head->next;
        head->prev = nullptr;
    }
    delete temp;
    size--;
}
```

Listing 7. Usuwanie pierwszego elementu

18. Obsługa iteratora

W klasie wykorzystano obiekt iterator, który umożliwia przechodzenie do kolejnych i poprzednich elementów.

```
void ListIterator::next() {
    if (current != nullptr) current = current->next;
```

```
}
```

Listing 8. Przejście do następnego elementu

19. Czyszczenie listy

Metoda `clear()` usuwa wszystkie elementy listy. Przechodzi przez kolejne węzły zwalniając pamięć i zerując licznik.

```
void DoublyLinkedList::clear() {
    while (head != nullptr) {
        pop_front();
    }
}
```

Listing 9. Usuwanie wszystkich elementów listy

Wnioski Celem projektu było zaimplementowanie listy dwukierunkowej w języku C++ z wykorzystaniem wzorców projektowych Factory oraz Iterator. W ramach projektu utworzono klasy odpowiedzialne za przechowywanie elementów listy, zarządzanie jej zawartością oraz poruszanie się pomiędzy kolejnymi elementami.

Zaimplementowane operacje umożliwiają dodawanie i usuwanie elementów na początku oraz końcu. Program pozwala również na wyświetlanie zawartości listy, przechodzenie do następnego i poprzedniego elementu oraz usuwanie wszystkich elementów.

Podczas realizacji projektu wykorzystano wskaźniki i dynamiczne zarządzanie pamięcią. Istotnym elementem implementacji było prawidłowe aktualizowanie połączeń pomiędzy węzłami, aby zachować strukturę listy po wykonaniu operacji dodawania i usuwania. Zastosowanie destruktora oraz metody `clear()` umożliwia zwalnianie pamięci zajmowanej przez elementy listy.

Tab. 19.1. Podsumowanie funkcjonalności projektu

Funkcjonalność	**Opis**
Dodawanie elementów	Początek i koniec listy
Usuwanie elementów	Początek i koniec listy
Wyświetlanie listy	Kolejność od początku do końca
Iterator	Przejście do następnego i poprzedniego elementu
Czyszczenie listy	Usunięcie wszystkich elementów
NodeFactory	Tworzenie nowych węzłów listy
Zarządzanie pamięcią	Zwalnianie pamięci usuwanych węzłów

Spis rysunków

Spis tabel

19.1. Podsumowanie funkcjonalności projektu 11

Spis listingów

1. Klonowanie repozytorium 4
2. Deklaracja klasy Node 7
3. Konstruktor klasy Node 8
4. Konstruktor i destruktor klasy listy 8
5. Dodawanie elementu na początku listy 8
6. Wyświetlanie listy w przód 9
7. Usuwanie pierwszego elementu 9
8. Przejście do następnego elementu 9
9. Usuwanie wszystkich elementów listy 10